PLANO DE TESTES E CASOS DE TESTE

Módulo de autenticação, recuperação de senha e controle de acesso

Responsável: Danilo (QA) Data: 02/11/2026

Base: especificações da API de Autenticação e da API de Recuperação de Senha e Moradores fornecidas pela equipe.

1. Objetivo

Definir o escopo, a abordagem, os critérios e os casos de teste para verificar o login, a identificação do usuário autenticado, a recuperação de senha e as permissões por perfil. Os testes serão executados principalmente por requisições à API e, quando o protótipo estiver disponível, complementados pela conferência do comportamento visual do front-end.

2. Escopo dos testes

Incluído	Fora do escopo nesta rodada
Login com credenciais válidas e inválidas; validação dos campos; resposta e dados retornados.	Recuperação por SMS/WhatsApp; renovação automática do JWT; bloqueio de login após várias tentativas.
Consulta do usuário autenticado por GET /api/v1/auth/me; ausência, adulteração e expiração do token.	Avaliação de desempenho/carga e teste de invasão completo.
Esqueci a senha e redefinição com token válido, inválido, expirado ou reutilizado; regras de senha e limite de pedidos.	Verificação visual definitiva do protótipo, pois ele não foi fornecido para esta versão do documento.
Permissões para SINDICO, MORADOR e PORTEIRO nas rotas descritas; distinção entre 401 e 403.	Funcionalidades descritas como futuras, como cobranças e recursos de portaria ainda não implementados.

3. Tipos de teste

Tipo	Aplicação
Funcional	Validar respostas de sucesso e erro dos endpoints de autenticação e recuperação.
Validação de dados	Campos obrigatórios, formato de e-mail, confirmação de senha e força mínima da nova senha.
Segurança básica	Não revelar se um e-mail existe; não retornar senha/hash; rejeitar tokens inválidos, vencidos ou adulterados.
Autorização	Confirmar permissões conforme perfil e negar ações não autorizadas.
Integração	Verificar a sequência de solicitação do link, recebimento do token e redefinição da senha; validar login com a senha nova.
Consistência documental	Comparar endpoints, campos, mensagens e permissões entre as duas especificações; comparar com protótipo quando disponível.

4. Ambiente e dados de teste

Ambiente: API de desenvolvimento/homologação, quando disponibilizada. Base URL documentada: /api/v1. Requisições e respostas em JSON. Para rotas protegidas, enviar Authorization: Bearer .

Usuários de teste descritos na especificação: sindico@condominio.com, ana@condominio.com (MORADOR) e portaria@condominio.com (PORTEIRO), com senha de teste 123456. Usar somente em ambiente de teste; não usar credenciais reais.

Para recuperação, utilizar uma caixa de e-mail de teste ou mecanismo de inspeção de e-mail/log disponibilizado pela equipe. Se o envio real ainda não estiver implementado, registrar a limitação e validar o token pelo mecanismo de desenvolvimento acordado.

5. Critérios de entrada e saída

Entrada	Saída
Especificações da API disponíveis; ambiente acessível; usuários de teste preparados; endpoints implementados ou identificados como pendentes.	Todos os casos prioritários executados; resultados registrados no Jira; falhas acompanhadas de passos, resultado esperado/obtido e evidência; divergências documentais comunicadas à equipe.

Critério de aprovação: o resultado observado deve corresponder ao resultado esperado descrito em cada caso. Caso a funcionalidade não esteja implementada ou o ambiente não permita executá-la, marcar como Bloqueado/Pendente, e não como Aprovado.

6. Convenções de resultado

200/201: operação concluída conforme o endpoint. 400: dados inválidos ou token de redefinição inválido conforme especificação. 401: autenticação ausente/inválida ou credenciais inválidas. 403: usuário autenticado sem permissão. 429: limite de solicitações de recuperação excedido. 5xx: falha inesperada do servidor.

7. Casos de teste

Os casos abaixo foram derivados das especificações recebidas. O campo “Status” deve ser preenchido durante a execução: Não executado, Aprovado, Reprovado ou Bloqueado.

CT01 — Login válido

Categoria: Autenticação Prioridade: Alta

Pré-condições: Usuário ativo existente; credenciais válidas.

Passos: 1. Enviar POST /api/v1/auth/login com email ana@condominio.com e senha 123456.

Resultado esperado: 200 OK; resposta contém token e usuario com id, nome, email e perfil MORADOR; unidade presente para MORADOR; senha/hash não aparecem na resposta.

Resultado obtido/evidência: preencher durante a execução no Jira.

CT02 — Login com senha incorreta

Categoria: Autenticação Prioridade: Alta

Pré-condições: Usuário existente.

Passos: 1. Enviar POST /api/v1/auth/login com email válido e senha incorreta.

Resultado esperado: 401 NAO_AUTENTICADO; mensagem “E-mail ou senha inválidos.”

Resultado obtido/evidência: preencher durante a execução no Jira.

CT03 — E-mail não cadastrado

Categoria: Autenticação Prioridade: Alta

Pré-condições: E-mail de teste inexistente.

Passos: 1. Enviar POST /api/v1/auth/login com e-mail inexistente.

Resultado esperado: 401 NAO_AUTENTICADO com a mesma mensagem usada para senha incorreta; a resposta não deve revelar se o e-mail está cadastrado.

Resultado obtido/evidência: preencher durante a execução no Jira.

CT04 — Campos de login ausentes/inválidos

Categoria: Validação Prioridade: Alta

Pré-condições: API disponível.

Passos: 1. Enviar POST /api/v1/auth/login com corpo vazio; repetir com e-mail malformado e/ou senha vazia.

Resultado esperado: 400 VALIDACAO; mensagem “Dados inválidos.” e detalhes dos campos inválidos.

Resultado obtido/evidência: preencher durante a execução no Jira.

CT05 — Consultar usuário autenticado

Categoria: Autenticação Prioridade: Alta

Pré-condições: Token JWT válido de usuário de teste.

Passos: 1. Enviar GET /api/v1/auth/me com Authorization: Bearer <token>.

Resultado esperado: 200 OK; dados correspondem ao dono do token; não há senha nem hash na resposta.

Resultado obtido/evidência: preencher durante a execução no Jira.

CT06 — Acessar rota protegida sem token

Categoria: Autenticação Prioridade: Alta

Pré-condições: Endpoint protegido disponível.

Passos: 1. Enviar GET /api/v1/auth/me sem cabeçalho Authorization.

Resultado esperado: 401 NAO_AUTENTICADO; mensagem indicando que o token não foi enviado.

Resultado obtido/evidência: preencher durante a execução no Jira.

CT07 — Token adulterado ou expirado

Categoria: Autenticação Prioridade: Alta

Pré-condições: Token inválido/adulterado ou token expirado preparado em ambiente de teste.

Passos: 1. Enviar GET /api/v1/auth/me com token adulterado; executar novamente com token expirado.

Resultado esperado: 401 NAO_AUTENTICADO; mensagem “Token inválido ou expirado.”

Resultado obtido/evidência: preencher durante a execução no Jira.

CT08 — Perfil sem permissão

Categoria: Autorização Prioridade: Alta

Pré-condições: Token válido de MORADOR.

Passos: 1. Enviar POST /api/v1/avisos com token de MORADOR.

Resultado esperado: 403 SEM_PERMISSAO; a ação não é executada e o usuário continua autenticado.

Resultado obtido/evidência: preencher durante a execução no Jira.

CT09 — Perfil autorizado

Categoria: Autorização Prioridade: Média

Pré-condições: Token válido de SINDICO.

Passos: 1. Enviar POST /api/v1/avisos com dados válidos e token de SINDICO.

Resultado esperado: Conforme a especificação de autenticação, 201 e aviso criado.

Resultado obtido/evidência: preencher durante a execução no Jira.

CT10 — Morador tenta cancelar reserva de outra pessoa

Categoria: Autorização por propriedade Prioridade: Alta

Pré-condições: Duas reservas pertencentes a moradores diferentes; token do primeiro morador.

Passos: 1. Enviar DELETE /api/v1/reservas/:id usando o identificador da reserva de outro morador.

Resultado esperado: 403 SEM_PERMISSAO; a reserva de outra pessoa permanece inalterada.

Resultado obtido/evidência: preencher durante a execução no Jira.

CT11 — Solicitar recuperação com e-mail cadastrado

Categoria: Recuperação Prioridade: Alta

Pré-condições: E-mail de teste cadastrado; acesso ao e-mail/log de teste.

Passos: 1. Enviar POST /api/v1/auth/esqueci-senha com e-mail cadastrado.

Resultado esperado: 200 OK com mensagem genérica; link/token de redefinição é emitido pelo mecanismo de e-mail definido para o ambiente.

Resultado obtido/evidência: preencher durante a execução no Jira.

CT12 — Solicitar recuperação com e-mail inexistente

Categoria: Recuperação Prioridade: Alta

Pré-condições: E-mail de teste não cadastrado.

Passos: 1. Enviar POST /api/v1/auth/esqueci-senha com e-mail inexistente.

Resultado esperado: 200 OK com a mesma mensagem genérica do CT11; não deve revelar que o e-mail não existe.

Resultado obtido/evidência: preencher durante a execução no Jira.

CT13 — Exceder limite de recuperação

Categoria: Recuperação Prioridade: Média

Pré-condições: Contador de tentativas limpo ou janela de uma hora controlada.

Passos: 1. Enviar quatro solicitações de recuperação para o mesmo e-mail dentro de uma hora.

Resultado esperado: A quarta solicitação excedente retorna 429 MUITAS_TENTATIVAS.

Resultado obtido/evidência: preencher durante a execução no Jira.

CT14 — Redefinir senha com token válido

Categoria: Recuperação Prioridade: Alta

Pré-condições: Token de redefinição válido, ainda não utilizado.

Passos: 1. Enviar POST /api/v1/auth/redefinir-senha com token válido, novaSenha com ao menos 8 caracteres contendo letra e número, e confirmacaoSenha igual.

Resultado esperado: 200 OK; mensagem de sucesso; token fica inutilizado; login com a senha nova funciona e com a antiga falha.

Resultado obtido/evidência: preencher durante a execução no Jira.

CT15 — Reutilizar token de redefinição

Categoria: Recuperação Prioridade: Alta

Pré-condições: Token já utilizado no CT14.

Passos: 1. Repetir POST /api/v1/auth/redefinir-senha com o mesmo token.

Resultado esperado: 400 TOKEN_REDEFINICAO_INVALIDO; senha não é alterada novamente.

Resultado obtido/evidência: preencher durante a execução no Jira.

CT16 — Token de redefinição expirado/inexistente

Categoria: Recuperação Prioridade: Alta

Pré-condições: Token expirado ou aleatório inexistente.

Passos: 1. Enviar POST /api/v1/auth/redefinir-senha com token inválido ou expirado e os demais campos válidos.

Resultado esperado: 400 TOKEN_REDEFINICAO_INVALIDO; mensagem genérica de link inválido ou expirado.

Resultado obtido/evidência: preencher durante a execução no Jira.

CT17 — Senha fraca ou confirmação divergente

Categoria: Recuperação Prioridade: Alta

Pré-condições: Token válido para testar validação; se a API validar campos antes do token, registrar a resposta correspondente.

Passos: 1. Enviar novaSenha com menos de 8 caracteres ou sem letra/número; repetir com confirmacaoSenha diferente.

Resultado esperado: 400 VALIDACAO com detalhes dos campos que falharam; senha não é alterada.

Resultado obtido/evidência: preencher durante a execução no Jira.

CT18 — Morador tenta listar moradores

Categoria: Permissões Prioridade: Alta

Pré-condições: Token válido de MORADOR.

Passos: 1. Enviar GET /api/v1/moradores com token de MORADOR.

Resultado esperado: A especificação de Recuperação e Moradores espera 403 SEM_PERMISSAO. Há divergência com a matriz geral de autenticação, que prevê consulta restrita à própria unidade; confirmar a regra antes de fechar o resultado.

Resultado obtido/evidência: preencher durante a execução no Jira.

CT19 — Porteiro tenta cadastrar morador

Categoria: Permissões Prioridade: Alta

Pré-condições: Token válido de PORTEIRO.

Passos: 1. Enviar POST /api/v1/moradores com dados válidos e token de PORTEIRO.

Resultado esperado: 403 SEM_PERMISSAO; nenhum morador é cadastrado.

Resultado obtido/evidência: preencher durante a execução no Jira.

CT20 — Requisição de moradores sem login

Categoria: Permissões Prioridade: Alta

Pré-condições: Nenhum token.

Passos: 1. Enviar PATCH /api/v1/moradores/2 sem Authorization.

Resultado esperado: 401 NAO_AUTENTICADO; nenhuma alteração é feita.

Resultado obtido/evidência: preencher durante a execução no Jira.

CT21 — Síndico lista e edita morador

Categoria: Permissões Prioridade: Média

Pré-condições: Token válido de SINDICO; morador existente.

Passos: 1. Enviar GET /api/v1/moradores; depois PATCH /api/v1/moradores/2 com uma alteração válida.

Resultado esperado: GET retorna 200; PATCH retorna 200 com os dados atualizados.

Resultado obtido/evidência: preencher durante a execução no Jira.

CT22 — Desativar morador

Categoria: Permissões Prioridade: Alta

Pré-condições: Token de SINDICO e morador ativo.

Passos: 1. Enviar PATCH /api/v1/moradores/:id com ativo=false; tentar login com as credenciais do morador desativado.

Resultado esperado: PATCH retorna 200; login subsequente falha com 401, conforme especificação.

Resultado obtido/evidência: preencher durante a execução no Jira.

CT23 — E-mail repetido ao cadastrar morador

Categoria: Validação de moradores Prioridade: Média

Pré-condições: E-mail já utilizado por outro usuário.

Passos: 1. Enviar POST /api/v1/moradores com um e-mail já cadastrado e token de SINDICO.

Resultado esperado: 409 CONFLITO; o sistema não cria um segundo usuário com o mesmo e-mail.

Resultado obtido/evidência: preencher durante a execução no Jira.

CT24 — Morador inexistente

Categoria: Validação de moradores Prioridade: Média

Pré-condições: Token de SINDICO válido.

Passos: 1. Enviar GET /api/v1/moradores/999 para um ID inexistente.

Resultado esperado: 404 NAO_ENCONTRADO.

Resultado obtido/evidência: preencher durante a execução no Jira.

8. Conferência de consistência das especificações

O protótipo não foi disponibilizado nesta rodada; portanto, não é possível afirmar que as telas foram comparadas. Os pontos abaixo são divergências identificadas entre os dois documentos de API e devem ser confirmados com o responsável pelo produto/equipe.

Ponto	O que consta nos documentos	Ação recomendada
Recuperação de senha	A especificação de autenticação diz que recuperação por e-mail está fora do escopo; o documento de recuperação define POST /auth/esqueci-senha e POST /auth/redefinir-senha.	Atualizar a especificação de autenticação para incluir as rotas novas ou indicar que o documento de recuperação é a versão mais recente.
Permissão GET /moradores	A especificação de moradores diz que apenas SINDICO pode listar/consultar moradores; a matriz geral de autenticação prevê que MORADOR consulte somente a própria unidade e que PORTEIRO consulte alguns dados.	Definir qual regra vale para GET /moradores e GET /moradores/:id; atualizar a matriz e os testes de acordo.
Validade do JWT	O token tem validade de 8 horas, mas a própria especificação marca a confirmação desse prazo como decisão pendente do PO.	Confirmar o prazo com o PO antes de considerar o requisito definitivo.
Regras de senha	A recuperação propõe mínimo de 8 caracteres, com letra e número, e indica que o PO ainda deve confirmar a regra.	Confirmar o critério e mantê-lo igual na API e na tela.
Protótipo/front-end	O documento de autenticação pede que o front trate 401 apagando o token e retornando ao login, e que 403 mantenha a sessão; as telas não foram fornecidas para verificação.	Quando o protótipo estiver disponível, conferir mensagens, campos, navegação e tratamento de 401/403.

9. Conferência com o protótipo — checklist para completar

☐ A tela de login contém e-mail e senha e envia os campos com os nomes definidos pela API.

☐ Credenciais inválidas exibem mensagem genérica sem revelar se o e-mail existe.

☐ Campos inválidos exibem mensagens compatíveis com o erro 400 VALIDACAO.

☐ O link “Esqueci minha senha” leva ao fluxo de solicitação de recuperação.

☐ A tela de redefinição recebe token, nova senha e confirmação; valida os requisitos acordados.

☐ Após redefinir a senha, o usuário volta para a tela de login; não ocorre login automático.

☐ Após erro 401 em rota protegida, o front remove o token e volta ao login.

☐ Após erro 403, o front mantém a sessão e informa que o usuário não tem permissão.

10. Modelo de registro de defeito

Campo	Preenchimento
Título	[QA] Resumo objetivo do problema
Ambiente	Desenvolvimento/homologação, navegador ou ferramenta e versão, se aplicável
Pré-condições	Usuário, perfil, token/dados necessários (não registrar senha real nem token sensível)
Passos para reproduzir	Passo 1; passo 2; passo 3...
Resultado esperado	Comportamento descrito na especificação/caso de teste
Resultado obtido	O que aconteceu de fato, incluindo status HTTP e mensagem
Evidência	Print, resposta da API ou log sem dados sensíveis
Severidade/prioridade	Definir com a equipe conforme impacto

11. Observações finais

Este documento é uma base inicial de planejamento. Os casos devem ser atualizados se o PO resolver as divergências de permissão, prazo do token ou política de senha. Os testes de interface permanecem pendentes até que o protótipo esteja disponível. Durante a execução, registrar evidências e abrir bugs

conforme o modelo acima.